

# Day 10: Pointers & References for Hardware Access

---

Platform: nRF52840 | RTOS: Zephyr RTOS

---

## Learning Objectives

---

### Pointer Fundamentals

A **pointer** stores the memory address of another variable. In embedded, pointers are how you access hardware registers, pass large data efficiently, and build data structures:

```
uint32_t value = 42UL;
uint32_t *ptr  = &value; // ptr holds the address of value
*ptr = 100UL;           // Dereference: write through the pointer
```

### volatile Pointer for MMIO

When accessing a **memory-mapped hardware register**, the compiler must not cache the read — the hardware can change the register value at any time. Both the pointer and the pointed-to value can need

`volatile`:

```
// The register value is volatile (hardware can change it)
volatile uint32_t *uart_status = (volatile uint32_t *)0x40002108UL;
uint32_t status = *uart_status; // Always reads from the actual address

// Both pointer and value are volatile (rare – pointer in shared memory)
volatile uint32_t * volatile dma_ptr;
```

### const Pointer vs Pointer to const

This is one of C++'s most confusing distinctions — master it:

```
const uint8_t *p;          // Pointer to CONST data – cannot modify *p, but can move p
uint8_t * const p;         // CONST pointer – can modify *p, but cannot move p
const uint8_t * const p;   // Both const – fully immutable
```

In embedded: `const uint8_t data` is used for read-only buffers (UART TX, SPI TX). `uint8_t const buf` is used when the pointer is fixed (e.g., a DMA buffer at a fixed address).

## References in Embedded

A **reference** is an alias for an existing variable. Unlike pointers, references:

- Cannot be null
- Cannot be re-bound (always refer to the same variable)
- Have no pointer arithmetic

```
void configure(volatile uint32_t &reg) {
    reg |= BIT(3);    // Modifies the actual register through the reference
}
```

Prefer references over pointers for function parameters when null is not a valid value.

## reinterpret\_cast for MMIO (C++ Style)

In C++, use `reinterpret_cast` instead of C-style casts for MMIO:

```
// C style (avoid in C++)
volatile uint32_t *uart_dr = (volatile uint32_t *)0x40002004UL;

// C++ style (preferred)
auto *uart_dr = reinterpret_cast<volatile uint32_t *>(0x40002004UL);
```

`reinterpret_cast` makes the dangerous cast explicit and searchable in code.

## extern "C" — C Linkage for Vendor Libraries

C++ **mangles** function names to encode types into the symbol name (for function overloading). Vendor SDKs written in C don't mangle names. Use `extern "C"` to tell the C++ compiler not to mangle a specific function:

```
extern "C" {  
    #include "nrfx_uart.h"    // Nordic HAL – C library  
}  
// OR for a single function:  
extern "C" void my_c_callback(void *ctx);
```

## nullptr vs NULL

In C++, always use `nullptr` instead of `NULL` or `0` for null pointers:

```
uint8_t *buf = nullptr;    // Type-safe null pointer (C++11)  
if (buf == nullptr) { /* safe */ }  
// nullptr has type std::nullptr_t – won't accidentally match int overloads
```

---

## Practice Tasks

1. Write a function that takes a `volatile uint32_t &` register ref and sets/clears bits
2. Access the nRF52840 GPIO output register directly via `reinterpret_cast` — toggle LED without Zephyr GPIO API
3. Create a read-only buffer `const uint8_t` and a writable buffer `uint8_t` — try (and fail) to write through the const one
4. Write an `extern "C"` wrapper that calls a C++ class method — useful for Zephyr C callbacks

---

## Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day10  
west build -b nrf52840dk/nrf52840 .  
west flash
```

```
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

## Resources

---

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day10/main.cpp](#)